# Dzanga MPI Parallel Simulation
# User Manual

**Monte Carlo Baseline • PPO Reinforcement Learning • Multicore Execution**

For dzanga_optimize_mpi.py and dzanga_sim_mpi.py

## 1. What must be installed?

MPI is not just a Python package. An MPI-enabled Python program normally uses two layers:

- MPI runtime/implementation — Open MPI, MPICH, or Microsoft MPI (MS-MPI). This supplies commands such as mpiexec and the native communication libraries.
- mpi4py — the Python interface to MPI. It lets Python code call MPI operations supplied by the installed MPI runtime.

**Therefore:** install an MPI runtime and install the Python package mpi4py. The other Python dependencies are NumPy, Pillow, Gymnasium, Stable-Baselines3, Matplotlib, and PyTorch.

| Component | Purpose | Example |
|---|---|---|
| MPI runtime | Starts and connects multiple processes | Open MPI / MPICH / MS-MPI |
| mpi4py | Python bindings to MPI | pip install mpi4py |
| Python ML stack | Simulation and PPO learning | stable-baselines3, torch, gymnasium |

## 2. Package contents

- dzanga_optimize_mpi.py — MPI-aware Monte Carlo, PPO training, evaluation, plotting, and smoke-test driver.
- dzanga_sim_mpi.py — Dzanga-Sangha simulation model.
- requirements_mpi.txt — Python dependencies including mpi4py.
- README_MPI.txt — concise command reference.

## 3. Installation - Ubuntu / Debian Linux

Open MPI is a straightforward choice on Ubuntu. From a terminal:

```
sudo apt update
sudo apt install -y openmpi-bin libopenmpi-dev python3-pip python3-venv
```

Create and activate a virtual environment (recommended):

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements_mpi.txt
```

Verify both layers:

```
mpiexec --version
python -c "from mpi4py import MPI; print(MPI.Get_version())"
```

## 4. Installation - Windows

On Windows, install an MPI implementation first. Microsoft MPI (MS-MPI) is commonly used. Install its runtime and SDK, then open a new Command Prompt or PowerShell window so PATH changes are visible. Alternatively, use an MPI distribution supported by your Python/mpi4py environment.

Then install the Python dependencies from the project directory:

```
py -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements_mpi.txt
```

Verify:

```
mpiexec
python -c "from mpi4py import MPI; print(MPI.Get_version())"
```

**Windows note:** If mpi4py installs but importing it reports that an MPI DLL/library cannot be found, the MPI runtime is missing, incompatible, or not on PATH. Reinstall/repair the MPI runtime and open a fresh terminal.

## 5. First tests

First confirm ordinary single-process execution. This helps distinguish simulation/Python problems from MPI problems:

```
python dzanga_optimize_mpi.py smoke --sim-seconds 20 --output smoke_serial
```

Then try two MPI processes:

```
mpiexec -n 2 python dzanga_optimize_mpi.py smoke --sim-seconds 20 --output smoke_mpi
```

After that, increase the process count to match the number of CPU cores/processes you want to use.

## 6. Parallel Monte Carlo baseline

The baseline is an excellent MPI workload because independent configuration/seed simulations can run simultaneously. Example with 8 MPI ranks:

```
mpiexec -n 8 python dzanga_optimize_mpi.py baseline --sim-seconds 86400 --dt 1 --seeds 20 --output
results
```

Interpretation: -n 8 launches eight MPI processes. --seeds 20 requests 20 Monte Carlo repetitions per configuration. --sim-seconds 86400 simulates one day per run, and --dt 1 advances the model in one-second simulation increments.

Useful shorter validation run:

```
mpiexec -n 4 python dzanga_optimize_mpi.py baseline --sim-seconds 3600 --dt 1 --seeds 4 --max-
cases 8 --output test_results
```

## 7. Parallel PPO reinforcement learning

For PPO, rank 0 manages the Stable-Baselines3 learner/model while MPI ranks execute independent environment simulations in parallel. This parallelizes rollout generation without allowing multiple processes to update the same neural-network optimizer independently.

```
mpiexec -n 8 python dzanga_optimize_mpi.py train --sim-seconds 86400 --dt 1 --timesteps 4096 --
output results
```

**Important:** --timesteps is the total PPO transition target. Adding MPI ranks increases the number of environments evaluated concurrently; it does not mean 4096 timesteps per rank.

## 8. Parallel evaluation of the trained PPO model

```
mpiexec -n 8 python dzanga_optimize_mpi.py evaluate --sim-seconds 86400 --dt 1 --seeds 100 --model
results/ppo_policy.zip --output results
```

Held-out random seeds are distributed across ranks, then rank 0 combines the evaluation results.

## 9. Plotting results

Plotting does not benefit materially from MPI. Run it normally after the baseline has completed:

```
python dzanga_optimize_mpi.py plot --output results
```

This reads the baseline summary and writes the comparison graphic into the output directory.

## 10. Choosing the number of MPI ranks

A good starting point is one MPI rank per physical CPU core, or somewhat fewer if memory use becomes significant. Do not assume that doubling ranks will exactly halve runtime: process startup, result gathering, memory bandwidth, and workload imbalance add overhead.

| Machine | Suggested first test |
|---|---|
| 4 cores | mpiexec -n 4 … |
| 8 cores | mpiexec -n 8 … |
| 16 cores | Start with -n 8, then compare -n 16 |
| Shared workstation/server | Leave cores available for the OS and other users when appropriate |

## 11. Output files

- baseline_runs.csv — individual Monte Carlo runs.
- baseline_summary.csv — aggregated baseline statistics.
- ppo_policy.zip — trained Stable-Baselines3 PPO model.
- training_config.json — simulation/training configuration.
- ppo_evaluation.csv — held-out PPO evaluation runs.
- chosen_configuration.json — deployment selected by PPO.
- ppo_deployment.csv — coordinates for the selected deployment.
- baseline_comparison.png — plotted baseline comparison.

## 12. Serial versus MPI commands

| Task | Serial | MPI example |
|---|---|---|
| Baseline | python ... baseline | mpiexec -n 8 python ... baseline |
| Train PPO | python ... train | mpiexec -n 8 python ... train |
| Evaluate | python ... evaluate | mpiexec -n 8 python ... evaluate |
| Plot | python ... plot | Prefer serial |

## 13. Troubleshooting

**mpiexec: command not found:** The MPI runtime is not installed or its executable directory is not on PATH.

**No module named mpi4py:** Activate the correct Python environment and run pip install mpi4py or pip install -r requirements_mpi.txt.

**Cannot load MPI library / DLL:** mpi4py is present but the native MPI runtime is missing or incompatible.

**Only one process appears to run:** Confirm the command begins with mpiexec -n N and that N is greater than 1.

**Program is slower with more ranks:** Use longer/more simulations; very small jobs can be dominated by MPI startup and communication overhead.

**PyTorch uses many threads per MPI rank:** MPI processes plus PyTorch/BLAS threads can oversubscribe the CPU. Consider setting OMP_NUM_THREADS=1 and MKL_NUM_THREADS=1 before launching many ranks.

**Output files conflict:** Only rank 0 should own shared aggregate output. Avoid manually launching multiple unrelated copies into the same output directory.

## 14. Recommended workflow

1. Run the serial smoke test.
2. Run a 2-rank MPI smoke test.
3. Run a small 4-rank baseline with --max-cases.
4. Run the full Monte Carlo baseline with an appropriate rank count.
5. Train PPO using MPI.
6. Evaluate the saved PPO model on many held-out seeds.
7. Compare PPO against the Monte Carlo baseline before drawing conclusions about superiority.

## 15. Example complete Linux session

```
source .venv/bin/activate

# Verify MPI
mpiexec --version
python -c "from mpi4py import MPI; print(MPI.Get_version())"

# Monte Carlo
mpiexec -n 8 python3 dzanga_optimize_mpi.py baseline --sim-seconds 86400 --dt 1 --seeds 20 --
output results

# PPO training
```

```
mpiexec -n 8 python3 dzanga_optimize_mpi.py train --sim-seconds 86400 --dt 1 --timesteps 4096 --
output results

# PPO evaluation
mpiexec -n 8 python3 dzanga_optimize_mpi.py evaluate --sim-seconds 86400 --dt 1 --seeds 100 --
model results/ppo_policy.zip --output results

# Plot
python3 dzanga_optimize_mpi.py plot --output results
```

## 16. Key architectural point

MPI is being used to parallelize independent simulation episodes, which is where this workload naturally exposes parallelism. The internal time steps of a single DzangaSimulation instance remain serial. For PPO, MPI workers perform environment rollouts while the Stable-Baselines3 PPO learner remains coordinated rather than creating competing copies of the optimizer.